

MOOD REFRESHER

Complete Project Report

A Stack-Based Python Command-Line Application

Student Name	Cheshta Maheshwari
Roll No.	26BCE11572
Semester	1
College	VIT BHOPAL
Faculty	Anjali Mathur
Academic Year	2026–27

Prepared according to the supplied Mood Refresher source code and project documentation.

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Project Overview
- 5. Technologies and Requirements
- 6. Project Structure
- 7. Functional Requirements
- 8. Non-Functional Requirements
- 9. System Architecture
- 10. Design Diagrams
- 11. Stack Data Structure and LIFO
- 12. Algorithms
- 13. Detailed Implementation
- 14. Mood-Based Recommendation System
- 15. Command-Line Execution
- 16. Error and Boundary Handling
- 17. Complexity Analysis
- 18. Strengths
- 19. Limitations
- 20. Future Scope
- 21. Results
- 22. Conclusion
- 23. References
- Appendix A · Source Code Excerpts
- Appendix B · Visual Evidence

1. Introduction

Mood Refresher is a Python-based command-line application developed around the Stack data structure. It allows a user to record one or more moods and then interact with those moods through a simple terminal dashboard. The latest mood is treated as the top of the Stack because the implementation follows the LIFO principle.

The project combines first-semester programming concepts: Python lists, functions, conditional statements, loops, input/output, string handling, and a practical implementation of Stack operations.

2. Problem Statement

- Accept mood entries from the user.
- Store multiple moods in a Stack.
- Identify and access the latest mood.
- Provide a predefined recommendation based on the mood.
- Display recorded moods from latest to oldest.
- Remove the latest recorded mood.
- Handle empty Stack conditions.
- Run completely from a terminal without a GUI-based setup.

3. Objectives

- Implement a Stack using a Python list.
- Demonstrate LIFO (Last In, First Out).
- Allow users to add multiple moods.
- Retrieve the latest mood using Peek().
- Remove the latest mood using pop().
- Display mood history using Display().
- Generate predefined mood-based recommendations.
- Demonstrate basic underflow handling.
- Provide a menu-driven command-line application.

4. Project Overview

The program starts with an empty list named Stack. The user enters moods one by one. Each mood is passed to Push(), which appends it to the list. The user types 'done' to finish the initial entry stage. The program does not proceed when 'done' is entered while the Stack is empty.

After at least one mood is recorded, the Advice Dashboard offers four operations: view advice for the latest mood, view all recorded moods, remove the latest mood, or exit.

5. Technologies and Requirements

Item	Project Choice
Programming language	Python 3.x
Application type	Command-line / terminal application

Item	Project Choice
Primary data structure	Stack
Stack implementation	Python list
External packages	None required
Execution environment	Terminal / Command Prompt / Shell

The project documentation specifies direct execution with `python mood_refresher.py` or `python3 mood_refresher.py`. Python's official documentation also supports running a script by supplying its filename to the interpreter.

6. Project Structure

```

.
├── mood_refresher.py
└── README.md

mood_refresher.py
    Complete Python implementation of Mood Refresher.

README.md
    Project description, features, requirements, setup, usage,
    Stack explanation and command-line execution instructions.
```

7. Functional Requirements

ID	Requirement	Implementation
FR-01	Mood Entry	Accept multiple mood entries until 'done'.
FR-02	Stack Storage	Push each entered mood into the Python-list Stack.
FR-03	Latest Mood Access	Peek retrieves the latest mood without removing it.
FR-04	Mood Advice	<code>get_mood_response()</code> generates predefined advice.
FR-05	Mood History	<code>Display()</code> prints moods from latest to oldest.
FR-06	Remove Latest	<code>pop()</code> removes the most recently added mood.
FR-07	Exit	Dashboard option 4 exits the program.
FR-08	Boundary Handling	Handles empty Stack and invalid menu choices.
FR-09	Dashboard Menu	Provides four choices for user interaction.
FR-10	Initial Validation	Does not allow 'done' before at least one mood is recorded.

8. Non-Functional Requirements

Category	Requirement
----------	-------------

Usability	Simple, menu-driven command-line interaction.
-----------	---

Portability	Runs in a standard Python 3.x terminal environment.
-------------	---

Dependencies	No third-party Python packages are required.
--------------	--

Performance	Push, Pop, Peek and isEmpty are $O(1)$; Display is $O(n)$.
-------------	--

Reliability	Handles empty Stack conditions and invalid dashboard selections with messages.
-------------	--

Maintainability	Different responsibilities are separated into functions.
-----------------	--

Execution Constraint	Runs directly from a terminal without GUI setup.
----------------------	--

Data Persistence	Mood history exists only during the current execution.
------------------	--

Input Behavior	Predefined mood categories are matched; other moods receive the default response.
----------------	---

9. System Architecture

User interacts through the Command-Line Interface. The main program controls mood entry and the dashboard. Stack functions manage mood entries, Display() shows history, and get_mood_response() selects predefined advice.

USER

v

Command-Line Interface (CLI)

v

Main Program & Dashboard Control

v

Stack (Python List) · LIFO

v

Push() | Peek() | pop() | isEmpty() | Display()

v

get_mood_response(mood_type)

v

Terminal Output / Advice

Layer / Component	Flow
get_mood_response(mood_type)	↓
Terminal Output / Advice	

10. Design Diagrams

10.1 Use Case Diagram

Actor	Use Case
USER	Enter / log moods
USER	View latest advice
USER	View mood history
USER	Remove latest mood
USER	Exit application

10.2 Workflow Diagram

```

START
↓
Initialize empty Stack
↓
Enter Mood
↓
Is input "done"?
  ■■■ NO → Push mood → Enter another mood
  ■■■ YES → Is Stack empty?
    ■■■ YES → Ask for at least one mood
    ■■■ NO → Advice Dashboard
      ■■■ 1 → Peek → Recommendation
      ■■■ 2 → Display → Mood History
      ■■■ 3 → Pop → Remove Latest
      ■■■ 4 → Exit

```

10.3 Sequence Diagram – View Latest Advice

Step	User	Dashboard	Stack / Advice
1	Selects 1	Receives choice	—
2	—	Requests latest mood	Peek()
3	—	Receives mood	Returns top item
4	—	Requests recommendation	get_mood_response()
5	—	Displays result	Returns advice

10.4 Component Diagram

Component	Responsibility	Relationship
Command-Line Interface	Input and output	Interacts with application logic
Dashboard Logic	Menu and workflow	Calls Stack / response functions
Stack Functions	isEmpty, Push, pop, Peek	Manages mood entries
get_mood_response()	Mood classification and advice	Receives a mood string
Display()	History display	Traverses recorded moods

11. Stack Data Structure and LIFO

A Stack is a linear data structure that follows LIFO (Last In, First Out). The last item inserted is the first item accessed or removed. Python lists provide `append()` and `pop()` operations that make them suitable for a simple Stack implementation.

Example:

Input order: happy → tired → excited

TOP



excited
tired
happy

Peek() → excited

pop() → removes excited

Next Peek() → tired

12. Algorithms

12.1 Algorithm for Adding a Mood

- Initialize an empty Stack.
- Ask the user for a mood.
- If input is 'done', check whether Stack is empty.
- If empty, ask for at least one mood.
- If not empty, proceed to dashboard.
- Otherwise call `Push(Stack, mood)`.
- Repeat until 'done'.

12.2 Algorithm for Viewing the Latest Mood

- Check whether Stack is empty.
- If empty, report that no mood is recorded.
- Otherwise call `Peek(Stack)`.
- Pass the mood to `get_mood_response()`.
- Display mood and recommendation.

12.3 Algorithm for Displaying Mood History

- Check whether Stack is empty.
- If empty, display the no-history message.
- Start from `len(Stack)-1`.
- Traverse backward to index 0.
- Print each mood in uppercase.

12.4 Algorithm for Removing the Latest Mood

- Call `pop(Stack)`.
- If return value is 'Underflow', report that Stack is empty.

- Otherwise display the removed mood.

13. Detailed Implementation

Function	Purpose	Code behavior
isEmpty(stk)	Checks Stack state	Returns True when stk == []
Push(stk, item)	Adds mood	Uses stk.append(item)
pop(stk)	Removes top mood	Returns Underflow if empty; otherwise stk.pop()
Peek(stk)	Reads top mood	Returns Underflow if empty; otherwise stk[len(stk)-1]
Display(stk)	Shows history	Loops from top index down to 0
get_mood_response(mood_type)	Chooses advice	Lowercases mood and uses if/elif/else

14. Mood-Based Recommendation System

Mood category	Recognized values	Behavior
Positive	happy, excited, joyful, great, calm	Celebration-oriented response.
Motivational	sad, tired, bored, uninspired, stressed, angry, fearless	Keep-going / breathe / song / water response.
Other	Any other value	General balance and focus response.

The supplied source contains 'angry' twice in the motivational list. Because membership testing is used, the duplicate does not create separate behavior.

15. Command-Line Execution

The project is designed to satisfy the requirement that it be fully executable through a terminal without a GUI-based setup.

```
python mood_refresher.py
```

or

```
python3 mood_refresher.py
```

```
=====
                WELCOME TO MOOD REFRESHER
=====
Lets log your moods first!

Enter your mood (or type 'done' to finish setting): happy
-> Logged: 'happy'
Enter your mood (or type 'done' to finish setting): tired
-> Logged: 'tired'
Enter your mood (or type 'done' to finish setting): done

All the moods stored successfully! Move to advice dashboard.....
```

Figure 1. Terminal-style startup and mood-entry flow based on the supplied implementation.

Dashboard Evidence

Visual evidence of the dashboard and latest-mood recommendation flow based on the supplied implementation.

```
=====
-----MOOD ADVICE DASHBOARD-----
=====
1. View advice for current/latest mood
2. View advice for ALL Recorded Moods
3. Remove unwanted/mistakenly added mood
4. Exit

Enter the choice (1-4): 1

---CURRENT MOOD:TIRED---
Advice KEEP GOING! Take a deep breath, play your favorite song and drink some water!
```

Figure 2. Dashboard and latest-mood recommendation flow based on the supplied code.

```
Stack after entry:

TOP -> tired
      happy

Option 2:
--- MOODS YOU ADDED ---
-Mood:TIRED
-Mood:HAPPY

Option 3:
Removed latest mood:'tired'

Stack now:
TOP -> happy
```

Figure 3. Display and Pop behavior based on the supplied Stack implementation.

16. Error and Boundary Handling

- isEmpty() is checked before Peek() and pop().
- Peek() returns 'Underflow' when Stack is empty.
- pop() returns 'Underflow' when Stack is empty.
- Initial entry does not allow 'done' before at least one mood is recorded.
- Dashboard choices outside 1–4 produce an invalid-choice message.
- The dashboard converts input using int(input(...)); a non-numeric menu entry can therefore raise ValueError.

17. Complexity Analysis

Operation	Time Complexity	Reason
Push	O(1)	append() adds at the end of the list.
Pop	O(1)	pop() removes the last list element.
Peek	O(1)	Direct access to the last element.
isEmpty	O(1)	Checks whether the list is empty.
Display	O(n)	Visits each recorded mood.

```
Mood Refresher — Invalid Menu Choice
Advice Dashboard
1. View advice  2. View moods  3. Remove latest  4. Exit
Enter choice: 7
Invalid choice! Please select between 1 and 4.
```

Invalid menu choice

```
Mood Refresher — Empty Stack Boundary
Advice Dashboard
3. Remove latest
Enter choice: 3
Underflow
```

Empty-Stack / Underflow

```
Mood Refresher — Non-numeric Menu Input
Enter choice: abc
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: 'abc'
```

Non-numeric input / ValueError

Figure 4. Examples of documented error/boundary behavior.

18. Strengths of the Project

- Clear demonstration of Stack operations using a Python list.
- Simple menu-driven terminal interface suitable for a first-semester project.
- Separate functions make the code easier to explain.
- Combines a practical theme with a core data-structure concept.
- Explicit empty-Stack handling is included.
- No third-party packages are required.

19. Limitations

- Command-line only; no GUI.
- Mood history is not persistent.
- No database or permanent file storage.
- Recommendations are fixed in source code.
- Recognized mood categories are limited.
- Non-numeric dashboard input can raise ValueError.
- Unrecognized moods receive the default response.

20. Future Scope

- Save mood history using a text file, CSV file, or database.
- Add timestamps.
- Add more mood categories and varied recommendations.
- Add mood frequency and trend statistics.
- Improve menu input validation.
- Add automated tests.
- Add a graphical interface in a future version.

21. Results

The implemented system demonstrates a working Stack-based command-line application. It can store multiple moods, retrieve the latest mood using `Peek()`, remove the latest mood using `pop()`, display moods in reverse insertion order, provide predefined recommendations, handle empty Stack cases, and run directly through a terminal.

22. Conclusion

Mood Refresher is a compact Python project demonstrating how a Stack can be applied to an interactive application. The LIFO principle makes the most recently entered mood the first mood accessed or removed. The project also demonstrates functions, lists, loops, conditional statements, string handling, input/output, basic error handling, and command-line execution. For a first-semester project, the implementation keeps the architecture small enough to understand while still showing a complete workflow.

23. References

1. Supplied Mood Refresher source file: mood_refresher.py.
2. Supplied Mood Refresher README/project documentation.
3. Python Documentation · Command Line and Environment.
4. Python Documentation · Data Structures / Using Lists as Stacks.
5. Project repository: <https://github.com/maheshwaricheshta740-ship-it/MoodRefresher.git>

Appendix A – Source Code Excerpts

A.1 Stack Implementation

```
def isEmpty(stk):
    if stk == []:
        return True
    else:
        return False

def Push(stk, item):
    stk.append(item)

def pop(stk):
    if isEmpty(stk):
        return "Underflow"
    else:
        return stk.pop()

def Peek(stk):
    if isEmpty(stk):
        return "Underflow"
    else:
        top = len(stk) - 1
        return stk[top]
```

A.2 Mood Response Function

```
def get_mood_response(mood_type):
    mood_type = mood_type.lower()

    if mood_type in ['happy', 'excited', 'joyful', 'great', 'calm']:
        return "CELEBERATION TIME! ..."

    elif mood_type in ['sad', 'tired', 'bored', 'uninspired',
                       'stressed', 'angry', 'fearless', 'angry']:
        return "KEEP GOING! ..."

    else:
        return "Stay balanced! ..."
```

The ellipses are used only to keep this appendix excerpt compact; the supplied source file remains the authoritative full implementation.

A.3 Dashboard Logic

```
if ch == 1:
    item = Peek(Stack)
    ...
elif ch == 2:
    Display(Stack)
elif ch == 3:
    item = pop(Stack)
    ...
elif ch == 4:
    print("Thank you for using MOOD REFRESHER! Have a great day!")
    break
else:
    print("Invalid choice! Please select between 1 and 4.")
```

Appendix B – Visual Evidence

These visuals are included to make the architecture and terminal behavior easier to understand. They represent the supplied command-line implementation; the project itself has no GUI.

```

=====
                WELCOME TO MOOD REFRESHER
=====
Lets log your moods first!

Enter your mood (or type 'done' to finish setting): happy
-> Logged: 'happy'
Enter your mood (or type 'done' to finish setting): tired
-> Logged: 'tired'
Enter your mood (or type 'done' to finish setting): done

All the moods stored successfully! Move to advice dashboard.....

```

Figure B1. Startup and mood-entry stage.

```

=====
-----MOOD ADVICE DASHBOARD-----
=====
1. View advice for current/latest mood
2. View advice for ALL Recorded Moods
3. Remove unwanted/mistakenly added mood
4. Exit

Enter the choice (1-4): 1

---CURRENT MOOD:TIREDD---
Advice KEEP GOING! Take a deep breath, play your favorite song and drink some water!

```

Figure B2. Advice Dashboard and Peek recommendation.

Stack after entry:

TOP -> tired
 happy

Option 2:

--- MOODS YOU ADDED ---

-Mood:TIREd

-Mood:HAPPY

Option 3:

Removed latest mood:'tired'

Stack now:

TOP -> happy

Figure B3. Stack history and Pop operation.